

Lời giải bài “Khống chế dịch bệnh”

Nguồn gốc: Epidemic Control solution report

IOI 2005 / Hu Weidong / attachments / Epidemic Control solution report.doc

1 Tóm tắt đề bài

Cho một cây, trong đó nút gốc 1 là bệnh nhân đã bị nhiễm. Trong mỗi chu kỳ lây lan, chỉ được cắt một cạnh của cây; sau đó mọi nút con của các nút đang bị nhiễm cũng sẽ bị nhiễm. Cần tìm một phương án cắt cạnh tốt nhất sao cho tổng số nút bị nhiễm là nhỏ nhất.

2 Phân tích lời giải

Gọi tập các nút bị nhiễm có độ sâu lớn nhất là **nguồn lây mới nhất**, ký hiệu S . Sau x chu kỳ lây lan, độ sâu của nguồn lây mới nhất là cố định, bằng x , nếu lấy độ sâu của gốc 1 là 0.

Từ đó có thể nghĩ đến một quy hoạch động theo tập:

$f(x, S)$ = số nút bị nhiễm nhỏ nhất sau x chu kỳ, khi nguồn lây mới nhất là S .

Từ $f(x, S)$, nếu $C(S)$ là tập tất cả các nút con của các nút trong S , và $v \in C(S)$ là nút ở đầu sâu hơn của cạnh bị cắt, thì trạng thái kế tiếp có dạng

$$f(x + 1, C(S) \setminus \{v\}).$$

Cách này nhìn có vẻ trực tiếp, nhưng số trạng thái theo tập là quá lớn; độ phức tạp bộ nhớ tăng theo cấp mũ của số nút trên một tầng, hoàn toàn không thể cài đặt.

Vì khó khăn về bộ nhớ, ta chuyển sang tìm kiếm. Xét không gian trạng thái: đề bài cho một cây. Gọi cây của đề bài là cây gốc; số tầng của cây tìm kiếm chính là độ sâu của cây gốc, còn số nhánh của cây tìm kiếm liên quan đến độ rộng của cây gốc. Nếu cây gốc rất sâu thì thường độ rộng nhỏ; nếu cây gốc rất rộng thì độ sâu lại nhỏ. Vì vậy quy mô tìm kiếm không lớn như cảm giác ban đầu.

3 Cách tìm kiếm

Trong x chu kỳ, bệnh luôn lan đến đúng độ sâu x . Nếu cắt một cạnh ở độ sâu nhỏ hơn x , bệnh đã đi qua cạnh đó nên việc cắt không còn ý nghĩa. Nếu cắt một cạnh ở độ sâu lớn hơn x , hiệu quả chắc chắn không bằng cắt cạnh nối đến tổ tiên tương ứng ở gần nguồn lây hơn. Do đó trong một phương án tối ưu, cạnh bị cắt ở chu kỳ hiện tại phải là một trong các cạnh nối từ nguồn lây mới nhất S đến các nút con của nó.

Tìm kiếm như vậy đã vượt qua được phần lớn dữ liệu, nhưng với dữ liệu cực hạn vẫn cần tối ưu thêm.

4 Các tối ưu

Tối ưu 1. Khi mới nhìn bài này, rất dễ nghĩ đến tham lam: mỗi lần chọn xóa cây con có nhiều nút nhất. Tuy cách tham lam này có phần ví dụ, nó vẫn là một hướng thử có xác suất cao dẫn tới nghiệm tốt sớm. Vì vậy khi duyệt các lựa chọn cắt cạnh, ta sắp các cây con theo kích thước giảm dần.

Tối ưu 2. Dùng cắt tia tối ưu. Giả sử hiện đang tìm kiếm đến tầng d , đã có *sick* nút bị nhiễm. Gọi *tmp* là tổng số nút con của các nguồn lây ở tầng d . Vì trong một chu kỳ chỉ cắt được một cạnh, ở chu kỳ kế tiếp sẽ có ít nhất $tmp - 1$ nút mới bị nhiễm. Do đó nếu

$$sick + tmp - 1 \geq ans,$$

ta có thể cắt nhánh tìm kiếm hiện tại.

Tối ưu 3. Khi tìm kiếm đến một hoặc hai tầng cuối, cấu trúc các cây con còn lại là như nhau đối với mục tiêu, nên chọn tùy ý một nhánh để xóa rồi thoát khỏi vòng lặp.

Sau khi thêm ba tối ưu trên, hiệu suất tăng lên rõ rệt. Dữ liệu chính thức lớn nhất được giải trong khoảng 0.2 giây theo báo cáo nguồn.

5 Chương trình tham khảo

Chương trình Pascal dưới đây được giữ nguyên từ nguồn gốc.

```
{R-,Q-,S-,I-}  
{M 65521,0,655360}  
{Author:Zhou Gelin}  
{Date:2005.1.11}  
{QQ:379688236}  
{MSN:zhougelin@hotmail.com}  
{Test Report}  
{P4 1.7G}  
{Windows Xp SP1}  
{Turbo Pascal 7.0}  
{  
EPIDEMIC.in1 = 10.0 (0.06s)  
EPIDEMIC.in2 = 10.0 (0.08s)  
EPIDEMIC.in3 = 10.0 (0.05s)  
EPIDEMIC.in4 = 10.0 (0.03s)  
EPIDEMIC.in5 = 10.0 (0.03s)  
EPIDEMIC.in6 = 10.0 (0.03s)  
EPIDEMIC.in7 = 10.0 (0.06s)  
EPIDEMIC.in8 = 10.0 (0.03s)  
EPIDEMIC.in9 = 10.0 (0.20s)  
EPIDEMIC.in0 = 10.0 (0.11s)  
}  
program epidemic;  
const inputfilename='epidemic.in';
```

```

        outputfilename='epidemic.out';
        maxn=300;
type pnode=^node;
    node=record
        data:integer;
        next:pnode;
    end;
    list=array[1..maxn]of integer;
var n,p,maxdep,ans:integer;
    g:array[1..maxn]of pnode;
    father:array[1..maxn]of integer;
    dep:array[1..maxn]of integer;
    sum:array[1..maxn]of integer;
    deg:array[1..maxn]of integer;
    son:array[1..maxn]of ^list;
    a:array[1..maxn]of boolean;
procedure insert(a,b:integer);
var q:pnode;
begin
    new(q);
    q^.data:=b;
    q^.next:=g[a];
    g[a]:=q;
end;
procedure read_data;
var i,a,b:integer;
begin
    fillchar(deg,sizeof(deg),0);
    assign(input,inputfilename);
    reset(input);
    readln(n,p);
    for i:=1 to p do
    begin
        readln(a,b);
        insert(a,b);
        insert(b,a);
        deg[a]:=deg[a]+1;
        deg[b]:=deg[b]+1;
    end;
    close(input);
end;
procedure dfs(v:integer);
var q:pnode;
    i,j,k:integer;
begin
    if father[v]>0 then
    begin
        deg[v]:=deg[v]-1;

```

```

    dep[v]:=dep[father[v]]+1;
    if dep[v]>maxdep then maxdep:=dep[v];
end
else dep[v]:=1;
getmem(son[v],2*deg[v]);
sum[v]:=1;q:=g[v];i:=0;
while q<>nil do
begin
    if q^.data<>father[v] then
    begin
        i:=i+1;
        son[v]^i:=q^.data;
        father[q^.data]:=v;
        dfs(q^.data);
        sum[v]:=sum[v]+sum[q^.data];
    end;
    q:=q^.next;
end;
for i:=1 to deg[v]-1 do
begin
    k:=i;
    for j:=i+1 to deg[v] do
        if sum[son[v]^j]>sum[son[v]^k] then k:=j;
    j:=son[v]^i;
    son[v]^i:=son[v]^k;
    son[v]^k:=j;
    end;
end;
end;
procedure search(deep,sick:integer);
var i,j,tmp:integer;
begin
    if deep=maxdep then
    begin
        if ans>sick then ans:=sick;
        exit;
    end;
    tmp:=0;
    for i:=1 to n do
        if a[i] and (dep[i]=deep) then
            tmp:=tmp+deg[i];
    if tmp=0 then begin search(maxdep,sick); exit; end;
    if sick+tmp-1>=ans then exit;
    for i:=1 to n do
        if a[i] and (dep[i]=deep) then
            for j:=1 to deg[i] do
                a[son[i]^j]:=true;
    for i:=1 to n do
        if a[i] and (dep[i]=deep) then

```

```

        for j:=1 to deg[i] do
        begin
            a[son[i]^j]:=false;
            search(deep+1,sick+tmp-1);
            a[son[i]^j]:=true;
            if deep=maxdep-1 then break;
        end;
    for i:=1 to n do
        if a[i] and (dep[i]=deep) then
            for j:=1 to deg[i] do
                a[son[i]^j]:=false;
            end;
    procedure answer;
    begin
        assign(output,outputfilename);
        rewrite(output);
        writeln(ans);
        close(output);
    end;
    var time:longint;
    begin
        time:=meml[64:108];
        read_data;
        dfs(1);
        ans:=1000;
        fillchar(a,sizeof(a),0);
        a[1]:=true;
        search(1,1);
        time:=meml[64:108]-time;
        writeln(time);
        answer;
    end.

```